

PYTORCH

From tensors to training loops

WHAT IS PYTORCH?

PyTorch is an open-source machine learning framework for building and training neural networks in Python.

- **2016 release:** Facebook AI Research introduced PyTorch with a Python-first, define-by-run approach.
- **Rapid adoption:** its flexibility made it a research favorite; Caffe2 was merged in with PyTorch 1.0 (2018).
- **Open governance:** since 2022, hosted by the PyTorch Foundation under the Linux Foundation.

Today it is one of the most widely used frameworks for deep learning research and production.

WHY PYTORCH?

Training a neural network requires repeated numerical computations, gradients, and parameter updates. PyTorch provides tools for all three while letting us define and change the computations.

- **Automatic differentiation** computes gradients through the operations we write.
- **Models and optimizers** provide reusable building blocks for training.
- **CPU and accelerator support** lets tensor computations run on different devices.
- **Flexible Python code** makes it possible to experiment with architectures, losses, and training procedures.

PyTorch is especially useful when the model or training procedure needs more control or flexibility.

FROM A SCIKIT-LEARN PIPELINE TO PYTORCH

In scikit-learn, a pipeline often bundles preprocessing and model fitting behind `fit()`.

In the repository's PyTorch workflow, we make those steps visible:

The underlying machine-learning ideas are familiar; PyTorch exposes more of the training process.

1. Split the data and fit preprocessing on **training data only**.
2. Convert the transformed data into tensors and batches.
3. Define the model, loss, and optimizer.
4. Repeat forward passes, gradient calculations, and parameter updates.
5. Evaluate on validation data and later assess the final choice on test data.

THE PYTORCH WORKFLOW

1. **Dataset:** defines how to retrieve an example and its target.
2. **DataLoader:** groups examples into batches for training or evaluation.
3. **Model:** maps inputs to outputs using learnable parameters.
4. **Loss and optimizer:** measure prediction error and update parameters.
5. **Training loop:** connects these components for repeated learning steps.

PyTorch gives us control over each step. In return, we are responsible for connecting them correctly.

TENSORS: THE DATA PYTORCH WORKS WITH

A **tensor** is a multidimensional array used for inputs, targets, parameters, and intermediate results.

Compatible shapes, data types, and devices are necessary for operations and losses to work as intended.

```
x = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])
print(x.shape)  # torch.Size([2, 3])
```

- **Shape** describes the dimensions: here, two observations and three features.
- **Data type** describes the values: features are typically `float32`; classification labels are typically `long`.
- **Device** identifies where computation happens, such as a CPU or GPU.
- **Vs. NumPy** arrays are CPU-only; tensors can also run on GPUs and track gradients.

DATASET: ACCESS INDIVIDUAL EXAMPLES

A **Dataset** defines how to retrieve an example and its target.

The Dataset gives the rest of the workflow a consistent way to access observations. A custom Dataset can also load or transform individual examples on demand.

- `__len__` returns the number of examples.
- `__getitem__` retrieves one example and its target by index.
- `TensorDataset` provides a convenient dataset when the data are already tensors.

```
training_data = TensorDataset(x_train, y_train)
x, y = training_data[0]
```

DATALOADER: MAKE BATCHES

A **DataLoader** retrieves examples from a Dataset and combines them into batches.

- Batches make training manageable and produce one gradient-based update per batch.
- `shuffle=True` changes training order across epochs; validation and test loaders usually do not need shuffling.
- The last batch may be smaller when the dataset size is not divisible by the batch size.

```
train_loader = DataLoader(
    training_data,
    batch_size=32,
    shuffle=True,
)

x_batch, y_batch = next(iter(train_loader))
```

NN.MODULE: DEFINE THE MODEL

The model

An **nn.Module** holds learnable parameters and defines the forward computation.

```
class Classifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3, 16)
        self.fc2 = nn.Linear(16, 2)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

model = Classifier()
```

How it works

- `nn.Linear` is a layer that learns weights and biases. `__init__` creates two of them, `fc1` and `fc2`; `nn.Module` registers their parameters automatically.
- `torch.relu` adds a nonlinear activation. `forward` applies `fc1`, then `torch.relu`, then `fc2`, mapping each three-feature observation to two output **logits**.

Calling `model(x_batch)` *runs* `forward`*; defining the model alone does not train it.*

NN.SEQUENTIAL: CHAIN LAYERS

The shortcut

nn.Sequential is a ready-made nn.Module that runs its layers in order: the same model, with less code.

```
model = nn.Sequential(
    nn.Linear(3, 16),
    nn.ReLU(),
    nn.Linear(16, 2),
)
```

How it works

- The layers are the same ones used in `Classifier`; `nn.ReLU` is the layer form of `torch.relu`.
- Layers run in the order listed: `nn.Linear`, `nn.ReLU`, `nn.Linear`, ending in two output **logits**.
- Use `nn.Sequential` when data flows straight through the layers; write a custom `nn.Module` when `forward` needs branches, skip connections, or other flexible logic.

Calling `model(x_batch)` *still runs the forward pass; defining the model alone does not train it.*

LOSS FUNCTION: MEASURE PREDICTION ERROR

The **loss function** turns predictions and targets into a scalar objective for training.

```
loss_fn = nn.CrossEntropyLoss()
logits = model(x_batch)
loss = loss_fn(logits, y_batch)
```

For the two-class example:

- Logits have shape `(batch_size, 2)`; integer class labels have shape `(batch_size,)`.
- Pass **raw logits** to `CrossEntropyLoss`; apply softmax separately when probabilities are needed.
- The loss supplies the objective whose gradients guide the model's updates.

AUTOGRAD: COMPUTE GRADIENTS

The backward pass

PyTorch's **autograd** records operations involving tensors that require gradients.

```
loss.backward()
```

What to know

- The backward pass applies the chain rule to calculate gradients of the loss with respect to model parameters.
- Gradients are stored on parameters in their `.grad` attributes.
- Gradients **accumulate** across backward calls, so a typical training step clears them first.

Computing gradients does not itself change the model's parameters; the optimizer performs that update.

OPTIMIZER: UPDATE PARAMETERS

An **optimizer** applies an update rule to the model's parameters using their gradients.

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
optimizer.zero_grad()  # Clear old gradients before a new backward pass
# Compute the loss and call loss.backward() here.
optimizer.step()       # Update parameters using the new gradients
```

- `model.parameters()` tells the optimizer which parameters to update.
- The learning rate controls the update scale; Adam also maintains optimizer state.
- Clearing gradients avoids unintentionally adding the current batch's gradients to previous ones.

THE TRAINING LOOP

```
model.train()

for x_batch, y_batch in train_loader:
    optimizer.zero_grad()               # Clear previous gradients
    logits = model(x_batch)             # Forward pass
    loss = loss_fn(logits, y_batch)     # Measure prediction error
    loss.backward()                     # Compute gradients
    optimizer.step()                    # Update parameters
```

The loop repeats these steps across batches and epochs. An **epoch** is one pass through the training loader; each batch produces one optimizer update.

EVALUATION: CHECK GENERALIZATION

- `model.eval()` changes the behavior of layers such as dropout.
- `torch.no_grad()` disables gradient recording, reducing unnecessary work.
- Fit preprocessing on training data, then apply the same transformation to validation and test data.
- Use validation results to compare settings; reserve the test set for final assessment.

```
model.eval()
with torch.no_grad():
    logits = model(x_val)
```

Evaluation shows how the current model performs on held-out examples and helps guide model selection.

SAVING AND THE REPOSITORY WORKFLOW

`state_dict()` holds a model's learned parameters. To predict later, rebuild the architecture and load them.

Call `eval()` before predicting.

```
# save the learned parameters
torch.save(model.state_dict(), "model.pt")

# later: rebuild, load, predict
model = Classifier()
model.load_state_dict(torch.load("model.pt"))
model.eval()
with torch.no_grad():
    pred = model(x).argmax(dim=1)
```

